

Module 5

Deep Learning (Deep Neural Networks)

Resources

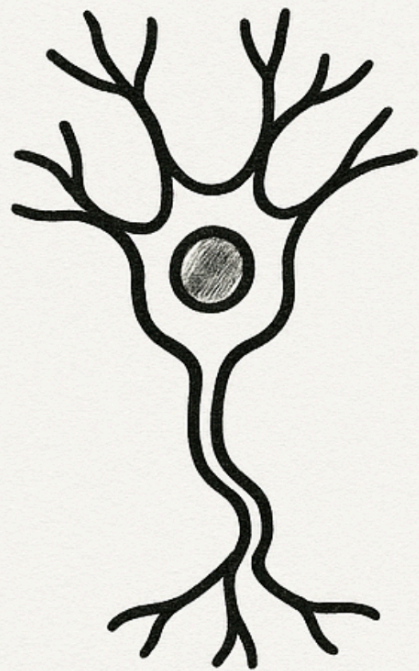
[PyTorch a 60-minute blitz](#)

Goodfellow et al. (2016) [Deep Learning](#)

Deep Neural Networks

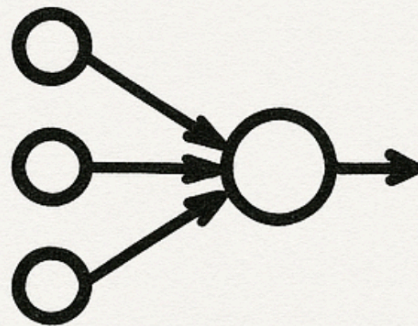
A very short history

A SHORT HISTORY OF NEURAL NETWORKS



1943

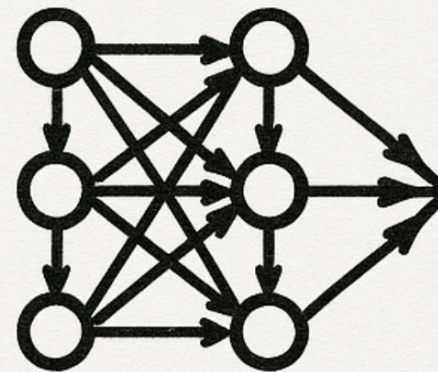
1943



McCULLOCH-
PITTS MODEL

1943

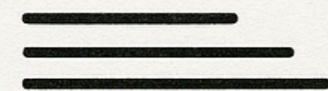
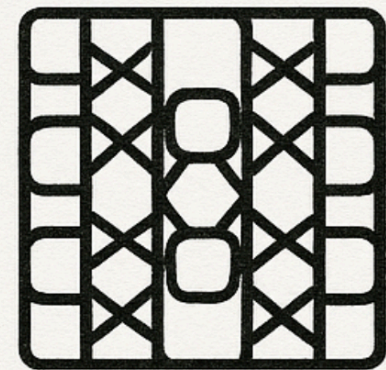
1980s



BACKPROP

2000s

2000s

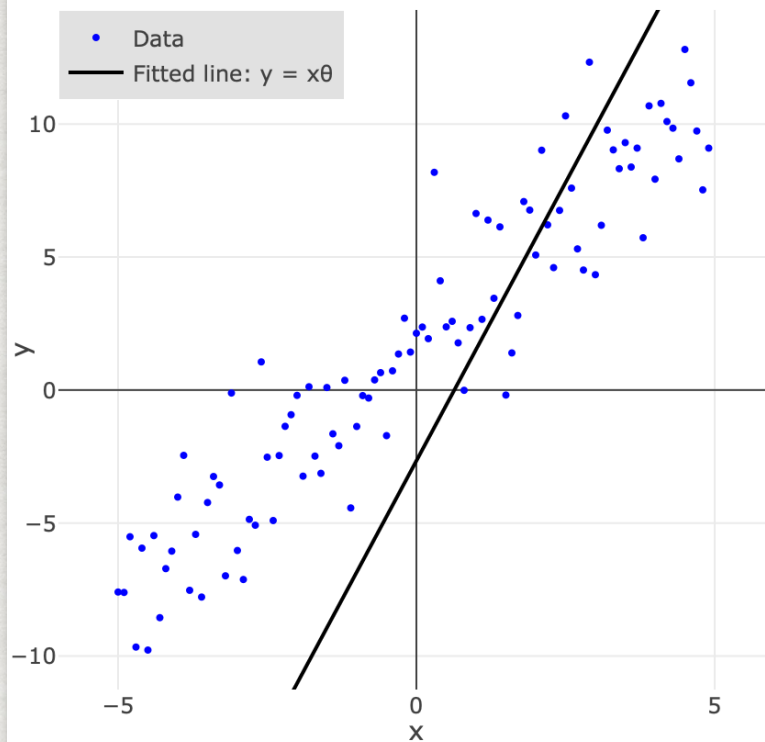


2010s

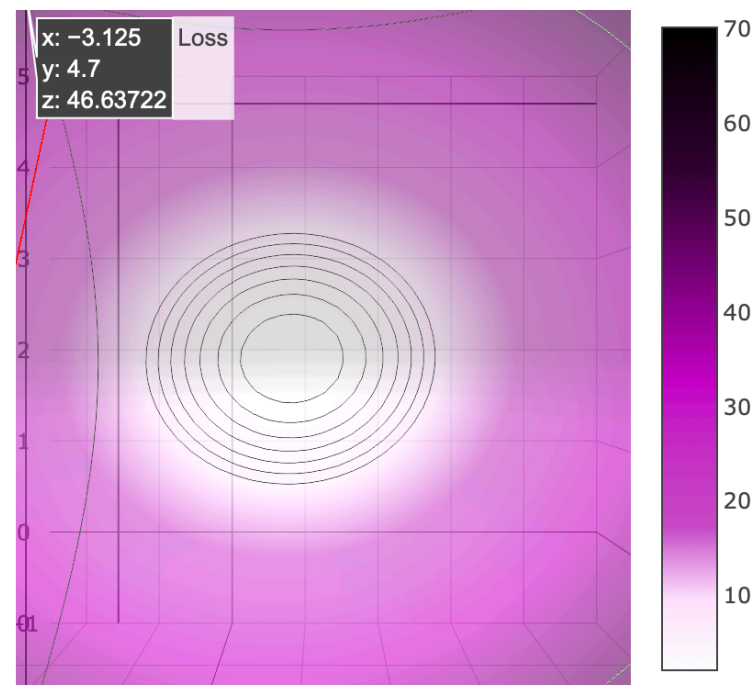
Learning through back-propagation

Gradient Descent

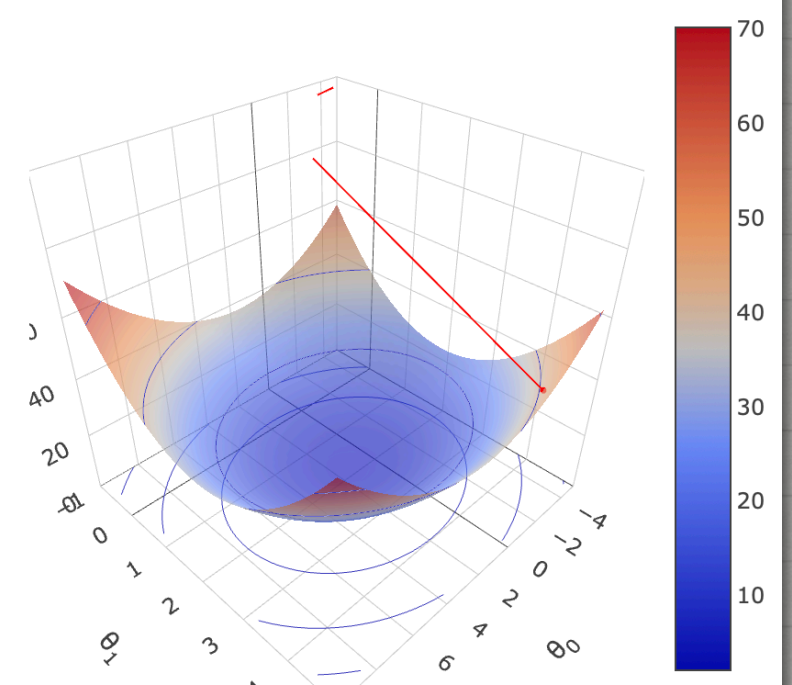
Labelled Data & Model Output



Contour Loss Function and Descent Path



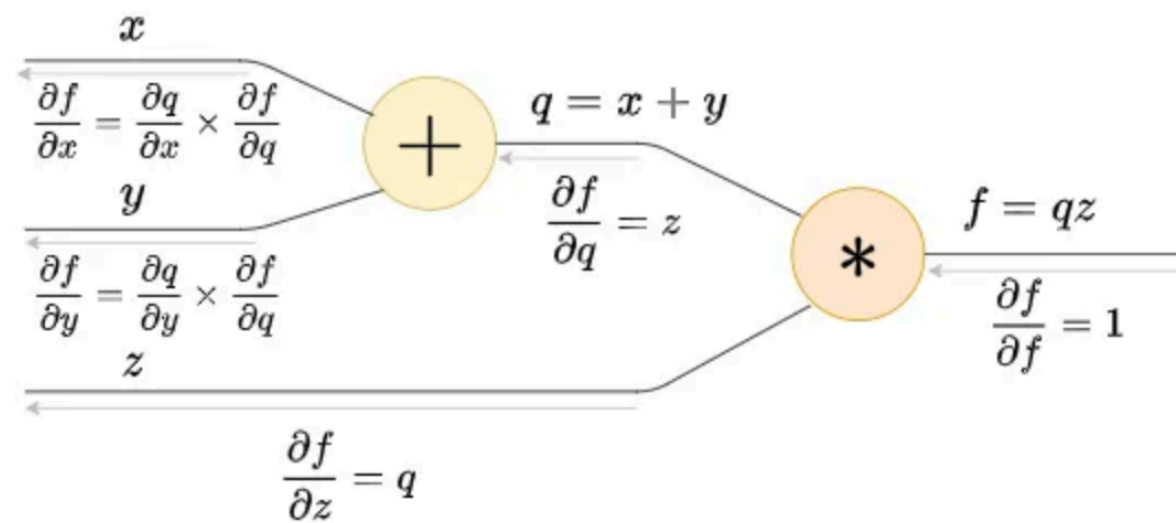
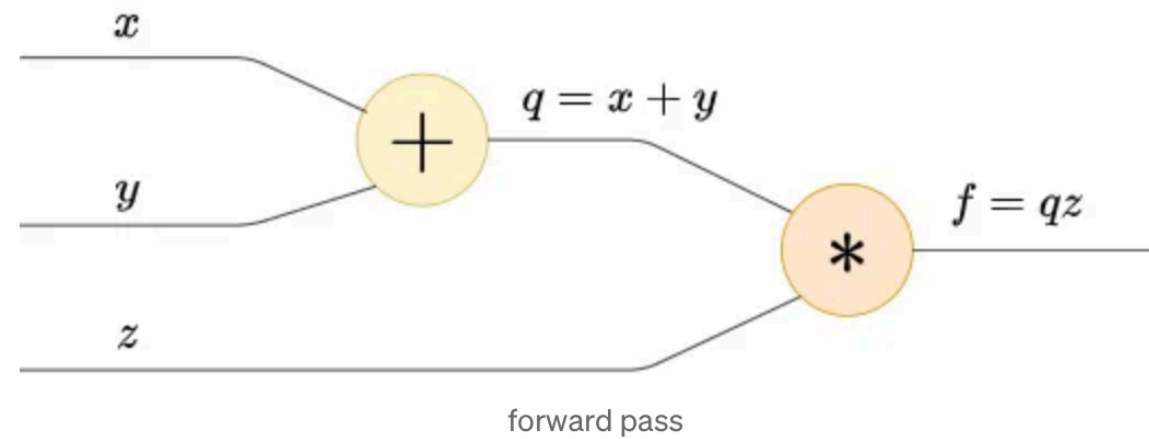
3D Loss Function and Descent Path



https://aero-learn.imperial.ac.uk/vis/Machine Learning/gradient_descent_3d.html

Learning through back-propagation

Credit assignment & backprop



The light gray arrows represent backward pass

Support for Deep Learning in Python

PyTorch:

```
$ conda activate cog-ai
```

```
$ pip install torch torchvision
```

Tensors (Like numpy arrays, but better):

```
x = torch.tensor([2.0, 3.0])
```

Autograd:

```
x = torch.tensor([2.0, 3.0], requires_grad=True)
```

torch.nn:

A library containing different types of NN layers and operations on those layers:

e.g. nn.Linear, nn.Sequential, nn.ReLU, nn.Conv2D, etc.

DataLoader:

An iterable utility that wraps a Dataset to provide an easy and efficient way to access data in batches. It handles tasks like automatically grouping samples into batches, shuffling the data each epoch, and parallelizing the data loading process across multiple worker processes

Mini-project 4: A small Neural Network

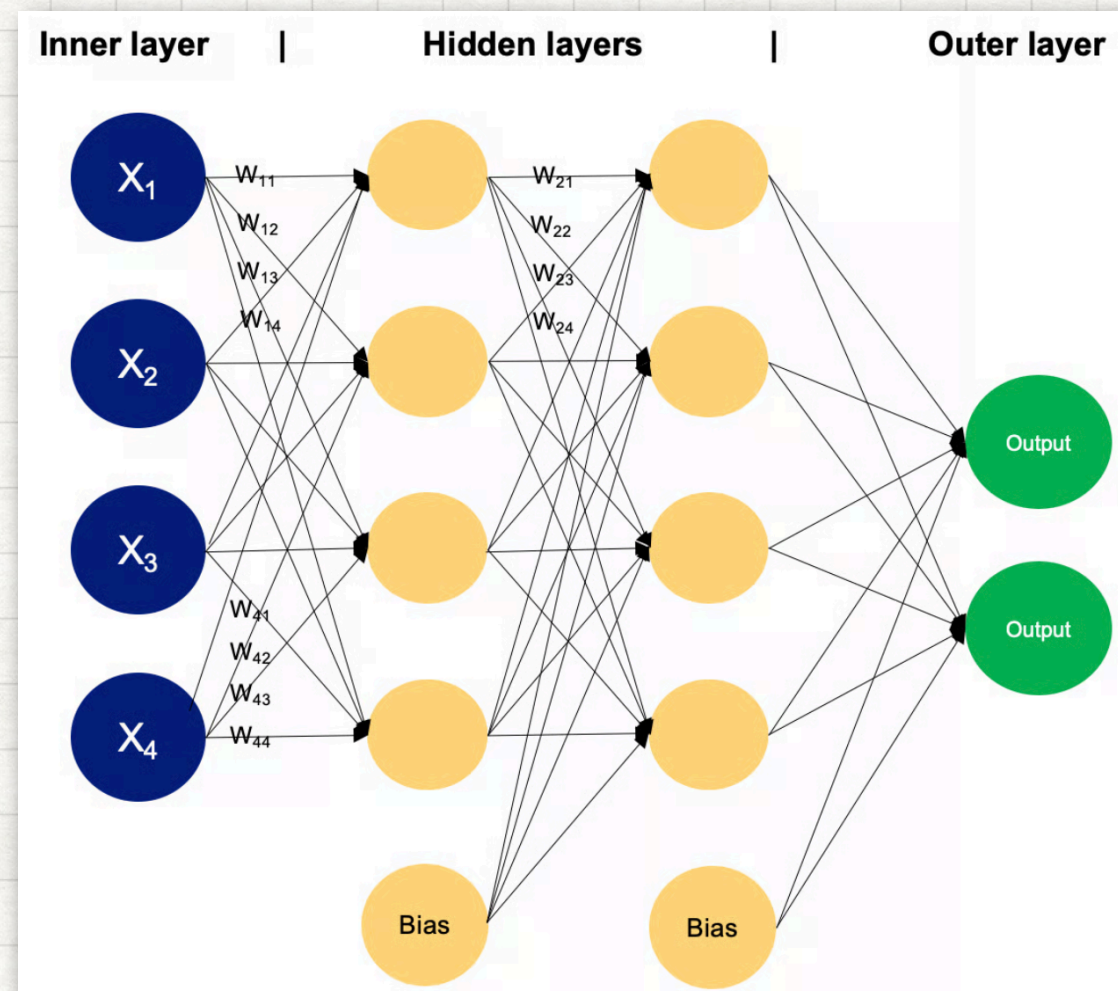
Train an MLP on MNIST and then analyse confusion patterns as a proxy for visual similarity.



MNIST Database

Mini-project 4: A small Neural Network

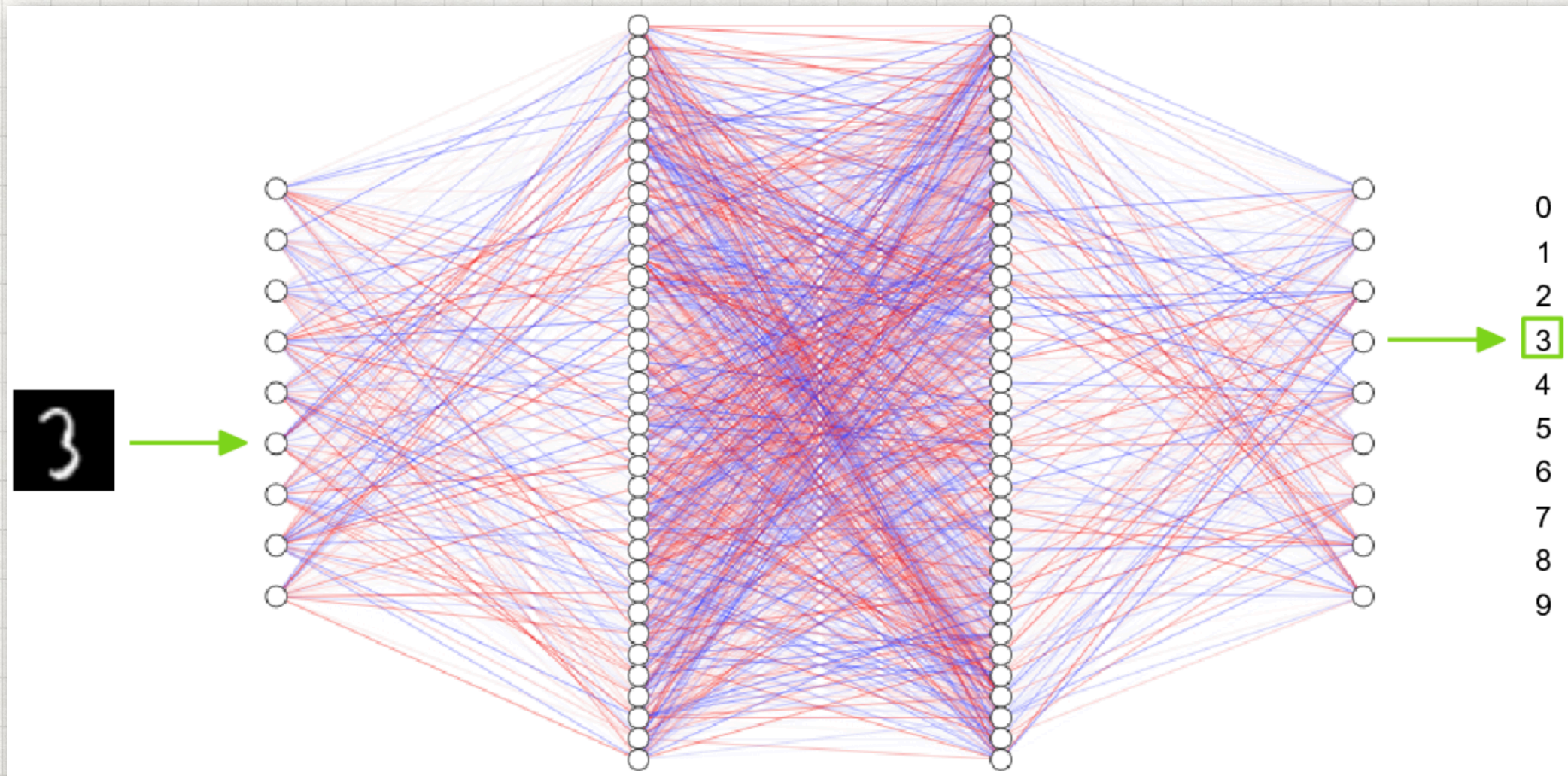
Train an MLP on MNIST and then analyse confusion patterns as a proxy for visual similarity.



Multi-layer Perceptron

Mini-project 4: A small Neural Network

Train an MLP on MNIST and then analyse confusion patterns as a proxy for visual similarity.



Demo 1: Using Tensors & Autograd

Define tensors 'a' and 'b'. Perform series of calculations on these tensors, using variables 'c', 'd', and 'e' to store outputs. Then print the gradient of 'e' with respect to 'a' and 'b'.

$$c = a + b$$

$$d = b + 1$$

$$e = c * d$$

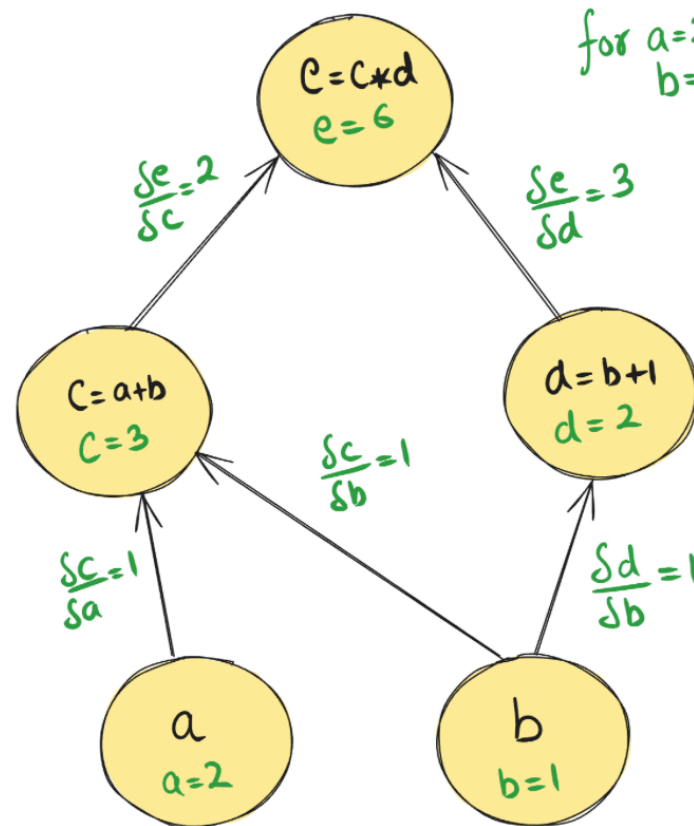
Compute: $\frac{\delta e}{\delta a}$ & $\frac{\delta e}{\delta b}$

Demo 1: Using Tensors & Autograd

Computational Graph

Computational Graph

Equation $e = (a * b)(b + 1)$ } can be written as
 $c = a + b$
 $d = b + 1$
 $e = c * d$



$$\frac{de}{da} = \frac{de}{dc} \times \frac{dc}{da} = 2 \times 1$$

$$\frac{de}{db} = \frac{de}{da} \times \frac{da}{db} + \frac{de}{dd} \times \frac{dd}{db} = 3 \times 1 + 2 \times 1 = 5$$

Demo 2: One Layer Network

Write Python code to program a one-layered network, that contains a single linear layer, with 'in_dim' inputs and 'out_dim' outputs. Give the network a random input, print the output. Also print the weights of the network.

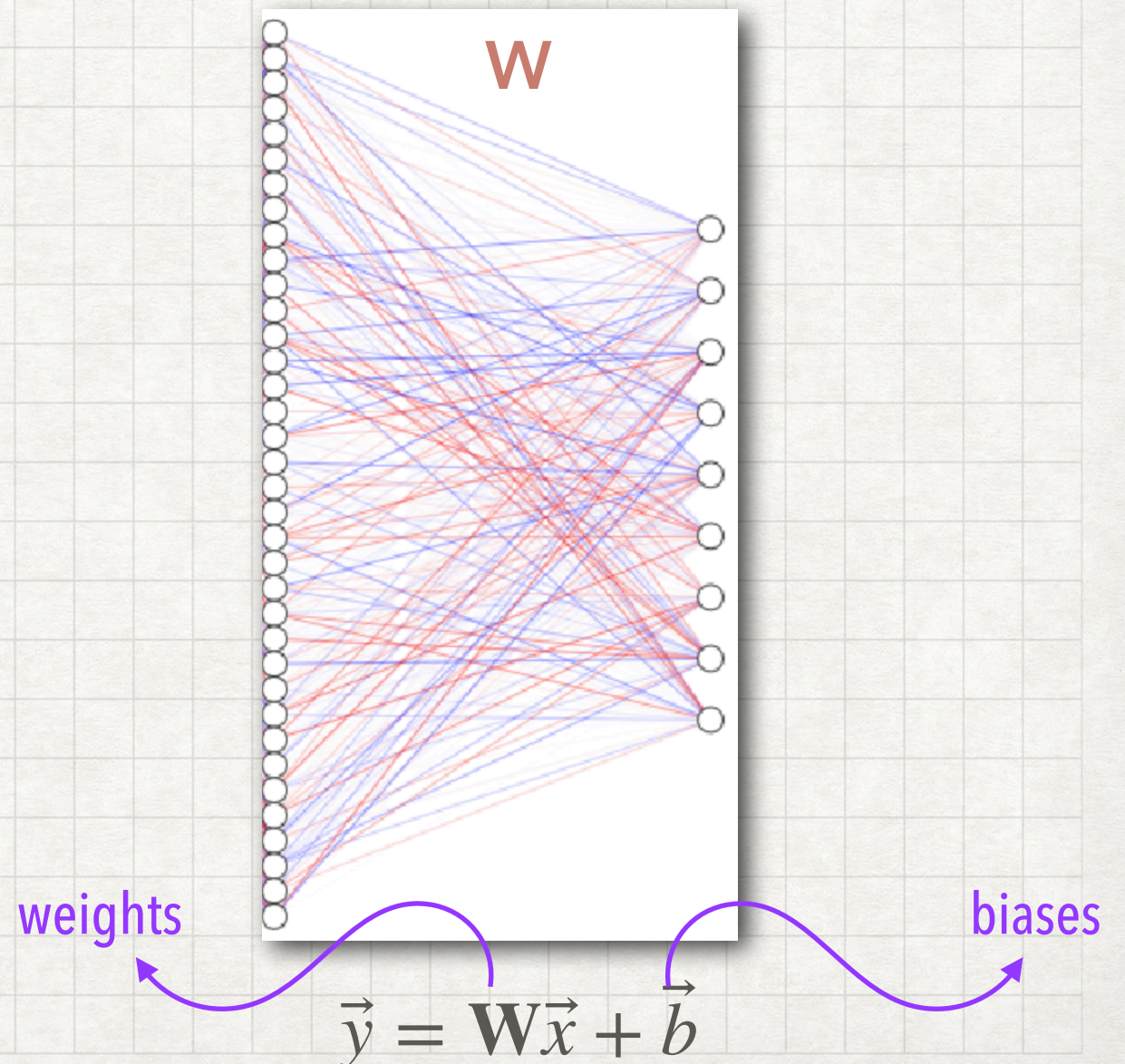
Pseudocode:

Define a class MyOneLayerdNet that:

- Create a Linear layer (matrix multiply + bias)
- Store it in self.fc

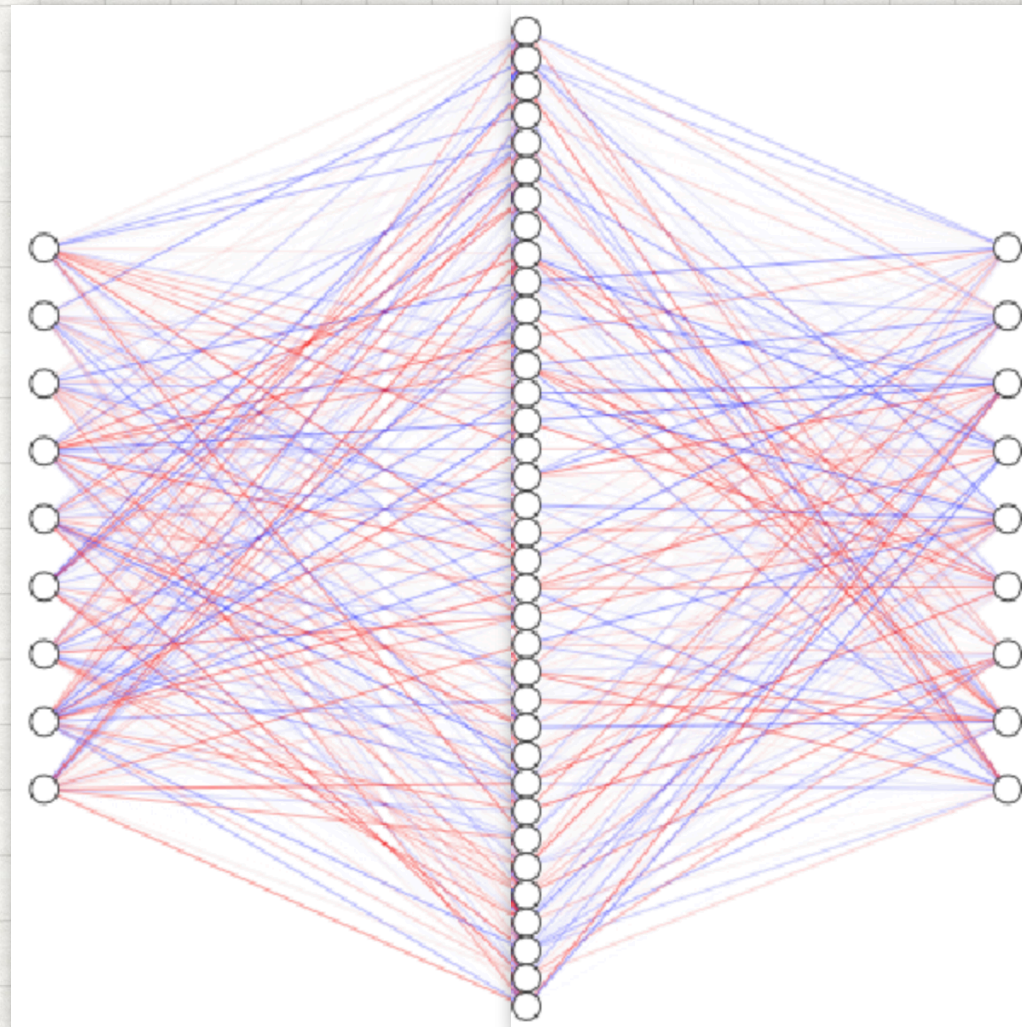
In forward(x):

- Apply the Linear layer to x
- Return the output



Demo 3: Small MLP

Write Python code to program a two-layered network, that contains a linear layer, followed by a ReLU function, which is then followed by another Linear layer. The output will be a set of "logits" (pre-softmax scores). Give the network a random input and see what the outputs are.



Demo 4: Learning

Write Python code to simulate one training step. This involves choosing a loss function, choosing an optimisation method (e.g. gradient descent), forward pass and change parameters based on gradients.

Pseudocode:

1. CREATE a small batch of training data
 - x : batch of input vectors; y : correct class labels
2. DEFINE a simple neural network
 - input → hidden; activation (ReLU); hidden → class scores
4. CHOOSE a loss function
 - CrossEntropyLoss (handles softmax + log loss)
5. CHOOSE an optimizer
 - SGD / Adam optimizer
 - connect it to the model's trainable parameters
6. ONE TRAINING STEP:
 - a. CLEAR old gradients
 - b. FORWARD PASS
 - feed input x through the model
 - get logits (raw class scores)
 - c. COMPUTE LOSS
 - compare logits with true labels y
 - d. BACKWARD PASS
 - autograd computes gradients of loss w.r.t. all model parameters
 - e. UPDATE PARAMETERS
 - optimizer uses gradients to adjust weights

Cross Entropy Loss

Entropy

$$H(X) = - \sum_i P(X = i) \log_2 P(X = i)$$

Cross-Entropy

$$H(t, p) = - \sum_{s \in S} t(s) \cdot \log(p(s))$$

Cross-Entropy
for DNN

$$H(p, q) = - \sum_{x \in \text{classes}} p(x) \log q(x)$$

True probability distribution (one-shot) $\nearrow$

$\nwarrow$ Your model's predicted probability distribution

<https://www.v7labs.com/blog/cross-entropy-loss-guide>

Demo 5: Load MNIST

Write Python code to load the MNIST dataset and then run one batch of images through the SmallMLP developed in previous demos. Print the logits for one of the images.

Pseudocode:

1. SET UP a transform for MNIST images
 - Convert each image to a tensor (values b/w 0 and 1)
2. LOAD the MNIST test dataset
 - Specify: use test split
 - Download if needed
 - Apply the transform to each image
3. WRAP the dataset in a DataLoader
 - Choose a batch size (e.g., 16 images per batch)
4. CREATE an instance of the MLP model
5. GET ONE BATCH from the DataLoader
 - images, labels = next batch from the iterator
 - images shape: (batch_size, 1, 28, 28)
 - labels shape: (batch_size)
6. RUN A FORWARD PASS THROUGH THE MODEL
 - logits = model(images)

Demo 6: In-class Activity

Can you put together **Demo 3** (how MLP is defined), **Demo 4** (one training step) and **Demo 5** (loading MNIST) to write code to train the network on MNIST. Here's some pseudocode:

1. PREPARE the MNIST DATA

- Define a transform: convert images to tensors
- Load training dataset (train=True)
- Wrap dataset in DataLoader:
 - train_loader: batch_size ~ 16, shuffle = True

2. CREATE model, loss function, and optimizer

- model is instance of SmallMLP
- Use CrossEntropyLoss for classification
- Use Adam optimizer on model parameters

3. TRAINING LOOP (for each epoch)

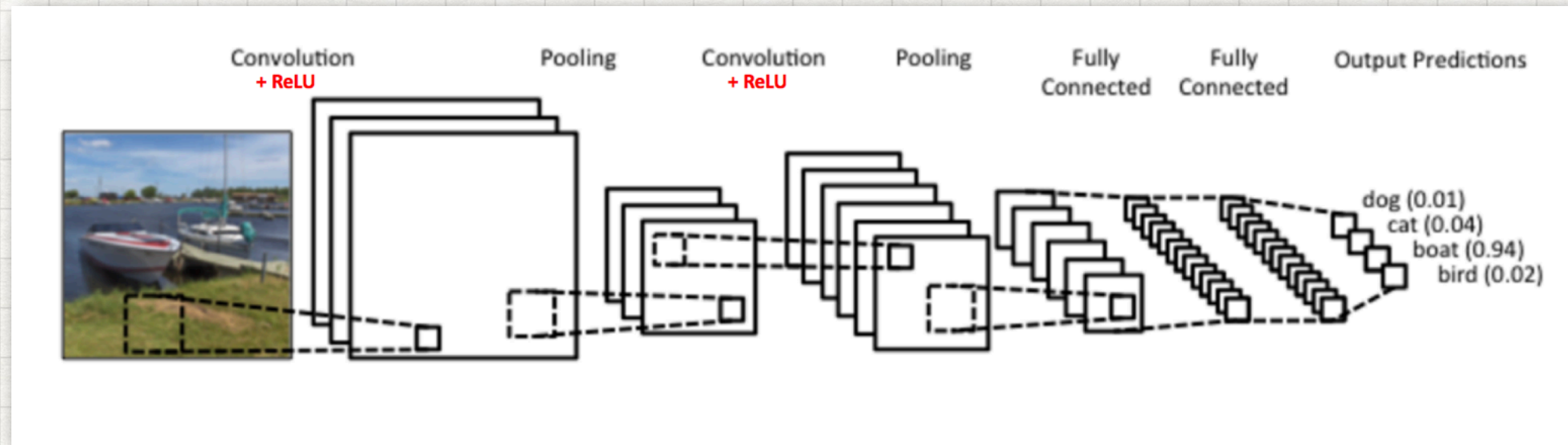
For epoch in 1..num_epochs:

- a. Set model to training mode
- b. Initialise running totals for loss and accuracy
- c. For each batch (x, y) in train_loader:
 - i. Move x and y to device
 - ii. Zero the optimizer gradients
 - iii. Forward pass: logits = model(x)
 - iv. Compute loss between logits and labels y
 - v. Backward pass: compute gradients of loss
 - vi. Optimizer step: update model parameters
 - vii. Update running loss and accuracy counters

d. At end of epoch:

- Compute average training loss
- Compute training accuracy
- Print epoch summary

Convolutional Neural Network



<https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/>

<https://youtu.be/oGpzWAlP5p0?si=rKoWKo98xV3t7DSF>

<https://cs231n.github.io/convolutional-networks/>